Program Optimization with Database-Style Term Rewriting

ANONYMOUS AUTHOR(S)

Since the 1970s, database engines have optimized queries by representing them in core calculi and repeatedly applying algebraic rewrite rules. Larger applications often treat database engines as important external data structures, benefiting from their internal optimizations. However, those applications are typically optimized with more conventional compiler techniques like dataflow analysis. We present here our experiences fusing application code with database logic and jointly optimizing them, now with algebraic rewrites for both, not just queries. That is, database-style data structures and algorithms are optimized with ahead-of-time knowledge of not just access patterns but also how query results are used in general-purpose code. The top new enabling idea is keeping track of contexts around queries using coercions between different consistency levels (list, bag, set). Our method is embodied in a library within the Rocq Prover, allowing rapid assembly of custom compiler pipelines with mechanized proof of correctness.

1 INTRODUCTION

Research in databases has led to intensive optimizations in query planners for relational database management languages, with SQL being a prominent example of such languages. Programmers can write declarative queries about the underlying database, and the query planner applies algebraic rewrite rules to determine the algorithmic details. We observe that, while many important applications are built on top of databases, typically the logic around database queries is written in a rather different style, and compilers optimize it in a rather different way. Is this divergence fundamental? Are there benefits to be gained by unifying the treatment of declarative queries and e.g. imperative loops over the results? We present one new approach in that direction, for programs that combine features of general-purpose languages with database-inspired comprehension code. Essentially, the compilers we build give programs custom in-memory databases, and we implement the whole framework in Rocq and generate compiler correctness proofs, an objective that is simplified by phrasing all optimization in terms of proved rewrite rules and orchestrating strategies for applying them.

Data-intensive applications nowadays often involve more complex logic than typical database queries, and it is unclear how to generalize the database-style term-rewriting framework to optimize such application code. Although procedural extensions to SQL have existed for a long time, writing complex application logic entirely in these extended SQL languages incurs significant performance cost, and many common programming-language features remain missing from the extensions [16]. It is therefore impractical to develop data-intensive applications entirely in SQL, so programmers naturally write code in an imperative rather than declarative style to handle the data returned from queries. As a result, there is need for explicit communication between application and database, and the misalignment between them causes the impedance mismatch problem [10]. The impedance mismatch causes overhead in both programming and execution due to the information flow across the two languages. Moreover, co-optimizations between the database representation and the application code should be possible if the programs that will run on the data are known at compile time [14], but the impedance mismatch makes it difficult to implement the optimizations correctly. In particular, SQL's default bag semantics can result in nondeterministic query outputs when a query retrieves multiple rows without specifying a total order [24]. Depending on how the query output is used, the application code may or may not behave nondeterministically. Hence, it becomes even harder to reason about the semantic preservation of optimizations on the application code. Prior works such as Links have explored integration of database management and application programming languages, but many choose to compile a subset of their languages to SQL [11]. That choice inherently ties the language semantics to SQL's semantics, but the SQL standard is

ambiguous, with different valid implementations of SQL potentially producing different results for the same query [24]. Furthermore, database-style optimizations are often nontrivial to implement, but most prior works provide no machine-checked proofs that their optimizations preserve program semantics.

This paper proposes to narrow the gaps though a framework built around the Conquord language, a language designed for data-intensive application programming, under the assumption that the queries that will be executed on the database are available at compile time. Conquord has language primitives common to imperative programming languages useful for writing application code, and it also has primitives that help express the interpretations of database tables as lists, bags, or sets. The source-to-source compiler for Conquord is capable of database-style optimizations such as applying relational-algebra rewrite rules. The compiler can also create auxiliary data structures known as indexes for a table and transform the application code to utilize the indexes for better runtime performance. Most notably, it identifies opportunities to relax the interpretation of a table, e.g. from a list to a bag, and open up new optimization possibilities without changing the meaning of the program. To reason about the soundness of the optimizations, we have a deep embedding of Conquord in the Rocq theorem prover, and Conquord programs have deterministic semantics. All the transformations in the compiler are formally verified to preserve the semantics of the programs.

One notable precursor is Fiat [13], a Rocq framework for derivation of programs from specifications by stepwise refinement. SQL-style queries are one natural form of specification, which can be handled in a unified framework based on a nondeterminism monad. As one proceeds translating a specification into a performant implementation, nondeterminism progressively collapses, for instance fixing the order of result rows coming out of a query to match what is easy to compute from the underlying data structure. On the one hand, working in a special monad adds complexity to a framework and brings compile-time costs in usability and performance, indicating the appeal of a framework that uses rewrites based on simple equality. On the other hand, when we consider information security, there are serious downsides of refinement in general.

In fact, it is widely known that refinement does not interact naturally with *information-flow* restrictions. As a concrete example, imagine that our data-intensive application is an email server, written out initially with queries and updates over tables representing different users' mailboxes. Let us also imagine a query to show a user's five most recent email messages, which is inherently ambiguous, given that a flood of messages might arrive at the same time. It is dangerous to give the compiler flexibility to resolve that ambiguity in any way compatible with refinement of nondeterministic programs. For instance, the choice of messages to show or the order to show them in might depend on consulting other users' mailboxes – e.g., Alice sees an order dependent on whether Bob has received messages from HotCRP about reviewing Alice's paper. That example is contrived and unlikely to arise in an honestly developed compiler, but we can imagine optimizations like sharing storage when multiple users receive the same message, which could lead to orders leaking information about email to other users.

A systematic solution is to commit to deterministic semantics of source programs, preserved by all compiler phases, but achieving good performance via algebraic rewrites becomes nonobvious. All prior approaches we are aware of either apply only to SQL-like code or are phrased using nondeterminism. We had to develop an important new design pattern for tracking *context* around queries, noting whether downstream code has the ability to distinguish differences in order or multiplicity. When downstream code is order-agnostic, we can return query results in the order most natural given the underlying data structure; while given more discriminating downstream code, we should sort the results. An alternative approach to drive such decisions would have been to rely on static analysis, perhaps with a type system, to track context. However, we found it aesthetically pleasing, and simplifying of our implementation, to track context using *casts* between

consistency levels (list, bag, set) and *pushdown* rules for casts, already familiar from classic database optimization.

1.1 A Motivating Example

Let us illustrate the approach with an example that introduces a dictionary-based index to accelerate a filter query. Our index maps keys to *bags* of values, which is safe only because the original query is invariant to the order of the values. Using bags instead of e.g. lists simplifies the concrete implementation of the index in a lower-level language.

Suppose we have two tables in the database: **departments** associates department IDs and department names, and **responses** stores survey results each consisting of an employee name, department ID, and feedback string. We write a Conquord program that joins the tables and produces a string containing the employee names, department names, and feedback:

```
letmut all-feedback := "" in
foreach r ←
  sort_list
  (r1 ← departments;
  r3 ← (r2 ← responses;
    check (r2.department-id == r1.department-id);
    ret r2);
  ret { name = r3.name; department = r1.department-name; feedback = r3.feedback }) do
all-feedback :=
  r.name ++ " from " ++ r.department ++
  " wrote: " ++ r.feedback ++ "\n" ++ all-feedback end
```

Now we show how this program is optimized by our system. The subexpression rooted at the sorting operation is parsed to:

```
sort_list
(flatmap_list departments (r1 →
  flatmap_list (flatmap_list responses (r2 →
    if (r2.department-id == r1.department-id) then [r2] else [])) (r3 →
      [{ name = r3.name; department = r1.department-name; feedback = r3.feedback }])))
```

After identifying relational-algebra operations (Section 4), this expression is:

```
sort_list
(flatmap_list departments (r1 →
  proj_list (filter_list responses (r2 → r2.department-id == r1.department-id)) (r3 →
    { name = r3.name; department = r1.department-name; feedback = r3.feedback })))
```

The sort operator means that this expression is invariant to the order of the results of the flatmaps. We take advantage of this property by introducing a bag-of cast that further rewriting will push

into the expression:

sort_{bag}
(bag-of ($\text{flatmap}_{\text{list}}$ **departments** ($r_1 \rightarrow$
  $\text{proj}_{\text{list}}$ ($\text{filter}_{\text{list}}$ **responses** ($r_2 \rightarrow r_2$.department-id == r_1.department-id)) ($r_3 \rightarrow$
    { name = r_3.name; department = r_1.department-name; feedback = r_3.feedback }))))

Collection-annotation-pushdown rules (Section 5) push the cast down to the **responses** table:

sort_{bag}
($\text{flatmap}_{\text{bag}}$ **departments** ($r_1 \rightarrow$
  proj_{bag} ($\text{filter}_{\text{bag}}$ (bag-of **responses**) ($r_2 \rightarrow$
    r_2.department-id == r_1.department-id)) ($r_3 \rightarrow$
      { name = r_3.name; department = r_1.department-name; feedback = r_3.feedback })))

The transformations above revealed the opportunity to create an index that is a dictionary mapping each department-id values to the bag of tuples in the **responses** table with the department-id attribute equal to that value. Index-creation rules (Section 6) create this index, store it in **responses**.aux, and rewrite the expression to use the index:

sort_{bag}
($\text{flatmap}_{\text{bag}}$ **departments** ($r_1 \rightarrow$
  proj_{bag} (dict-lookup (**responses**.aux) r_1.department-id) ($r_3 \rightarrow$
    { name = r_3.name; department = r_1.department-name; feedback = r_3.feedback })))

Rather than iterating through the entire **responses** table for every department-id value, the lookup operation retrieves each tuple from **responses** only once, so the final program is asymptotically faster than the original.

2 KEY IDEAS

From query comprehension to relational algebra. List-comprehension syntax is found in many programming languages including Python and Haskell, and variations of comprehension syntax have been proposed for database languages as an alternative to the SQL syntax [6, 27]. Query comprehensions can be parsed to monadic operations on lists, with flatmap as the monadic bind operator. We use query comprehensions in the concrete syntax of Conquord for its natural integration with the rest of the language, and flatmap is therefore a primitive of the Conquord language. After parsing the input program, the compiler identifies subexpressions that correspond to the filter, projection, and join operations in relational algebra. That normalization facilitates the subsequent database-style optimizations, which often involve relational-algebra operations.

For instance, the following expression is written in the query-comprehension syntax. It fetches each tuple from the **shirts** table and returns it in the output if its color attribute is "`white`".

$x \leftarrow$ **shirts**;
check (x.color == "`white`");
ret x

The query comprehension is parsed to the following abstract syntax tree.

flatmap **shirts** ($x \rightarrow$ if (x.color == "`white`") then [x] else [])

The compiler can then transform the expression (Section 4) to a semantically equivalent relational-algebra operation on **shirts**.

$$\text{filter } \textbf{shirts } (x \rightarrow x.\texttt{color} == \text{“}\texttt{white}\text{”})$$

Interpretation of collections. In the original relational-database theory, every relation is interpreted as a set of tuples [8]. SQL uses bag semantics by default, so a database table can contain duplicate tuples. However, the bag semantics permit nondeterministic outputs from SQL queries when the order is not specified, and the nondeterminism complicates the definitions of program equivalence and transformation soundness. To mitigate the problem, we use list semantics in the subset of the Conquord language exposed to the programmer. Nevertheless, some optimizations rely on the interpretation of a database table as a bag or a set of tuples, and interpreting database tables as lists everywhere in the program rules out such optimizations. The compiler recovers opportunities for alternative interpretations of collections by pattern-matching for certain local constructs. For example, sorting a list is equivalent to interpreting the list as a bag and sorting the bag. Such interpretations can then be propagated to operations on collections like flatmap, filter, and join.

Continuing from the example above, if the expression is under a sorting operation, then this stage of compilation will recognize that it can be interpreted as a bag. The expression

$$\text{sort}_{\text{list}} \ (\text{filter } \textbf{shirts } (x \rightarrow x.\texttt{color} == \text{“}\texttt{white}\text{”}))$$

gets transformed to

$$\text{sort}_{\text{bag}} \ (\text{bag-of } (\text{filter } \textbf{shirts } (x \rightarrow x.\texttt{color} == \text{“}\texttt{white}\text{”})))$$

and the bag annotation gets pushed (Section 5) into the subexpression to give

$$\text{sort}_{\text{bag}} \ (\text{filter}_{\text{bag}} \ (\text{bag-of } \textbf{shirts}) \ (x \rightarrow x.\texttt{color} == \text{“}\texttt{white}\text{”}))$$

Creating and using indexes. Creating the right indexes for a database table and using them for queries is an important way to improve the runtime efficiency of the queries. Many prior works have studied how to make better choices based on the queries and workload [1, 2, 5, 14, 15, 17, 20, 25, 26, 29, 30]. Rather than considering the heuristics used to choose the optimal combination of indexes, this work assumes the knowledge of a set of indexes that the compiler should create for a table. Such information can be derived through static analysis of the application code, and the choice of indexes will not compromise the soundness of the compilation pipeline as long as the pipeline is proven to be sound for any of the indexes it may create for a table. The Conquord compiler has a sequence of program transformations (Section 6) that create the indexes and rewrite the application code to use the created indexes, and the transformations together preserve the semantics of the programs.

One way to optimize the example program above is to use a dictionary-based index that maps each color value to the tuples in the **shirts** table where the color attribute takes that value. The filter operation can then be replaced by the corresponding lookup operation on the index. Since the lookup only needs to return a bag of tuples, the implementation of that index does not need to preserve the order of the tuples in **shirts**. As such, more optimization opportunities can arise due to the relaxed interpretation of collections.

Limitations. The design of the Conquord compiler assumes that all the application code that will run on the data is available at compile time and that the database tables are loaded fully into memory during program execution, so the compiler can automatically apply effective database-style optimizations based on the code. This paper focuses on the construction of such transformation pipelines with soundness proofs rather than the heuristics for choosing the optimal transformations.

Moreover, the transformations described in this paper are source-to-source transformations on the high-level Conquord language, and compilation to executable code is left for future work.

3 CONQUORD

In this section we describe the syntax and semantics of Conquord.

3.1 Types and Abstract Syntax

Conquord is a language with static type checking. Figure 1 shows the types in the Conquord language. A record type contains a list of pairs of tags (column names) and types sorted by a total order over the tags. They are well-formed only when tags are distinct. The semantics of a record expression are similar. Hence, the order of entries in a record expression does not matter, and the two well-typed expressions $\{\ \text{tag-1} = e_1;\ \text{tag-2} = e_2\ \}$ and $\{\ \text{tag-2} = e_2;\ \text{tag-1} = e_1\ \}$ have the same record type and value.

$$
\begin{array}{lrcl}
\text{Record type entry} & \mathcal{H} & ::= & \text{tag} = \mathcal{T} \\
\text{Type} & \mathcal{T} & ::= & \text{int} \mid \text{bool} \mid \text{string} \mid () \\
 & & & \mid \text{option}\ \mathcal{T} \mid \text{list}\ \mathcal{T} \mid \text{bag}\ \mathcal{T} \mid \text{set}\ \mathcal{T} \\
 & & & \mid \text{dict}\ \mathcal{T}\ \mathcal{T} \mid \{\ \mathcal{H}; \ldots\ \}
\end{array}
$$

Fig. 1. Conquord types

Figure 2 shows a fragment of the abstract syntax of Conquord. For brevity, we omit most unary and binary operations that are Conquord expressions. The different kinds of collections are built into the Conquord language design through a totally ordered set of three collection categories, which are used to annotate other primitives. The atoms in the Conquord language include integers n, Booleans b, and string literals s, as well as empty collections and some other common primitives. For brevity, we write $\text{empty}_{\text{list}}$ as $[]$ and $\text{insert}_{\text{list}}\ e_2\ e_1$ as $e_1 :: e_2$. The abstract syntax of the empty collections and the none expression permit an optional type annotation, which can be helpful for type inference in the bidirectional typechecker. Conquord expressions include both immutable variables and mutable variables. Mutable variables are useful for expressing updates to database tables, and we use bold symbols to denote them. The Conquord language is first-order, with the new variable binding x in the expression e expressed as $(x \rightarrow e)$. There is structural recursion through the fold operation on lists but no general recursion. Record construction, sorting, conversion from lists to bags and sets, and basic relational-algebra operations are also available as Conquord expressions, but the concrete syntax exposed to the programmer does not involve bags or sets. Conquord commands include common imperative constructs, but there is no while-loop, and the foreach constructs are guaranteed to terminate.

The construct flatmap_c for $c \in \{\text{list}, \text{bag}, \text{set}\}$ represents flat-map operations on the respective collections, likewise for other constructs with collection subscripts. In the rest of this paper, we will omit the subscript when referring to operations on lists.

3.2 Type System

Figure 3 shows the typing judgment for Conquord atoms. The type well-formedness judgment $\vdash t$ enforces the entries of each record type in t are ordered and there is no duplication among the tags in a record type.

The typing judgments on Conquord expressions are defined by inductive rules, some of which are shown in Figure 4. The typing environment for mutable variables is denoted by Δ, and that for immutable variables is denoted by Γ.

$$
\begin{array}{lcll}
\text{Collection} & \chi & ::= & \mathsf{set} \mid \mathsf{bag} \mid \mathsf{list} \\
\text{Record entry} & \mathcal{R} & ::= & \mathsf{tag} = \mathcal{E} \\
\text{Expr} & \mathcal{E} & ::= & x \mid \mathbf{x} \mid n \mid b \mid s \mid \mathsf{none} \mid \mathsf{none}^t \mid \mathsf{empty\text{-}dict} \\
 & & & \mid \mathsf{empty}_\chi \mid \mathsf{empty}^t_{chi}() \\
 & & & \mid \mathcal{E} < \mathcal{E} \mid \mathcal{E} + \mathcal{E} \\
 & & & \mid \mathsf{some}\ \mathcal{E} \mid \mathsf{length}\ \mathcal{E} \mid \mathsf{if}\ \mathcal{E}\ \mathsf{then}\ \mathcal{E}\ \mathsf{else}\ \mathcal{E} \\
 & & & \mid \mathsf{let}\ x \leftarrow \mathcal{E}\ \mathsf{in}\ \mathcal{E} \mid \mathsf{dict\text{-}insert}\ \mathcal{E}\ (\mathcal{E} \mapsto \mathcal{E}) \\
 & & & \mid \mathsf{sort}_\chi\ \mathcal{E} \mid \mathsf{sum}\ \mathcal{E} \mid \mathsf{count}\ \mathcal{E} \mid \mathsf{min}\ \mathcal{E} \\
 & & & \mid \mathsf{flatmap}_\chi\ \mathcal{E}\ (x \to \mathcal{E}) \mid \mathsf{fold}\ \mathcal{E}\ \mathcal{E}\ (x, y \to \mathcal{E}) \\
 & & & \mid \{\ \mathcal{R}; \ldots\ \} \mid \mathcal{E}.\mathsf{tag} \mid \mathsf{bag\text{-}of}\ \mathcal{E} \mid \mathsf{set\text{-}of}\ \mathcal{E} \\
 & & & \mid \mathsf{insert}_\chi\ \mathcal{E}\ \mathcal{E} \\
 & & & \mid \mathsf{filter}_\chi\ \mathcal{E}\ (x \to \mathcal{E}) \\
 & & & \mid \mathsf{join}_\chi\ \mathcal{E}\ \mathcal{E}\ (x, y \to \mathcal{E}\ \mathsf{if}\ \mathcal{E}) \\
 & & & \mid \mathsf{proj}_\chi\ \mathcal{E}\ (x \to \mathcal{E}) \\
\text{Command} & \mathcal{C} & ::= & \mathsf{skip} \mid \mathcal{C}; \mathcal{C} \mid \mathsf{letmut}\ \mathbf{x} \coloneqq \mathcal{E}\ \mathsf{in}\ \mathcal{C} \mid \mathbf{x} \coloneqq \mathcal{E} \\
 & & & \mid \mathsf{if}\ \mathcal{E}\ \mathsf{then}\ \mathcal{C}\ \mathsf{else}\ \mathcal{C} \mid \mathsf{foreach}\ x \leftarrow \mathcal{E}\ \mathsf{do}\ \mathcal{C}\ \mathsf{end}
\end{array}
$$

Fig. 2. Abstract syntax of Conquord

$$
\frac{}{\Delta; \Gamma \vdash n : \mathsf{int}} \qquad \frac{}{\Delta; \Gamma \vdash b : \mathsf{bool}} \qquad \frac{}{\Delta; \Gamma \vdash s : \mathsf{string}} \qquad \frac{}{\Delta; \Gamma \vdash () : ()}
$$

$$
\frac{\vdash t}{\Delta; \Gamma \vdash \mathsf{none} : \mathsf{option}\ t} \qquad \frac{\vdash t}{\Delta; \Gamma \vdash \mathsf{none}^t : \mathsf{option}\ t} \qquad \frac{\vdash kt \qquad \vdash vt}{\Delta; \Gamma \vdash \mathsf{empty\text{-}dict} : \mathsf{dict}\ kt\ vt}
$$

$$
\frac{\vdash t}{\Delta; \Gamma \vdash [] : \mathsf{list}\ t} \qquad \frac{\vdash t}{\Delta; \Gamma \vdash []^t : \mathsf{list}\ t} \qquad \frac{\vdash t}{\Delta; \Gamma \vdash \mathsf{empty}_{\mathsf{bag}} : \mathsf{bag}\ t} \qquad \frac{\vdash t}{\Delta; \Gamma \vdash \mathsf{empty}^t_{\mathsf{bag}} : \mathsf{bag}\ t}
$$

$$
\frac{\vdash t}{\Delta; \Gamma \vdash \mathsf{empty}_{\mathsf{set}} : \mathsf{set}\ t} \qquad \frac{\vdash t}{\Delta; \Gamma \vdash \mathsf{empty}^t_{\mathsf{set}} : \mathsf{set}\ t}
$$

Fig. 3. Typing judgment on Conquord atoms

The well-typedness judgments on Conquord commands can be defined using the typing judgments on Conquord expressions, as shown in Figure 5. We have developed a bidirectional type-checker and proven its soundness with respect to the typing judgments.

3.3 Big-Step Operational Semantics and Type-Soundness Theorem

Figure 6 shows the values used to define the semantics of Conquord expressions. Integer literals are denoted by n, Booleans by b, and strings by s. Like record types, a record value also has its entries sorted by the tags. Similarly, our implementation of dictionary, bag, and set values in Rocq ensures that equivalent values have unique and canonical representations.

Some of the typing rules on values are shown in Figure 7.

$$\frac{\Delta[\mathbf{x}] = t}{\Delta;\Gamma \vdash \mathbf{x} : t} \qquad \frac{\Gamma[x] = t}{\Delta;\Gamma \vdash x : t} \qquad \frac{\Delta;\Gamma \vdash e_1 : \text{bool} \qquad \Delta;\Gamma \vdash e_2 : t \qquad \Delta;\Gamma \vdash e_3 : t}{\Delta;\Gamma \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 : t}$$

$$\frac{\Delta;\Gamma \vdash e_1 : \text{list } t_1 \qquad \Delta;\Gamma \vdash e_2 : t_2 \qquad \Delta;\Gamma, x : t_1, y : t_2 \vdash e_3 : t_2}{\text{fold } e_1\ e_2\ (x, y \to e_3) : t_2}$$

$$\frac{\text{no-dup } [\text{tag-1}, \dots] \qquad \Delta;\Gamma \vdash e_1 : t_1 \qquad \dots}{\Delta;\Gamma \vdash \{ \text{ tag-1} = e_1, \dots \} : \{ \text{ tag-1} = t_1, \dots \}} \qquad \frac{\Delta;\Gamma \vdash e_1 : \text{list } t_1 \qquad \Delta;\Gamma, x : t_1 \vdash e_2 : \text{list } t_2}{\Delta;\Gamma \vdash \text{flatmap } e_1\ (x \to e_2) : \text{list } t_2}$$

$$\frac{\Delta;\Gamma \vdash e_1 : \text{bag } t_1 \qquad \Delta;\Gamma, x : t_1 \vdash e_2 : \text{bag } t_2}{\Delta;\Gamma \vdash \text{flatmap}_{\text{bag}}\ e_1\ (x \to e_2) : \text{bag } t_2} \qquad \frac{\Delta;\Gamma \vdash e : \text{bag } t}{\Delta;\Gamma \vdash \text{sort}_{\text{bag}}\ e : \text{list } t}$$

$$\frac{\Delta;\Gamma \vdash e : \text{list } t}{\Delta;\Gamma \vdash \text{bag-of } e : \text{bag } t} \qquad \frac{\Delta;\Gamma \vdash e_1 : \text{bag } t \qquad \Delta;\Gamma \vdash e_2 : t}{\Delta;\Gamma \vdash \text{insert}_{\text{bag}}\ e_1\ e_2 : \text{bag } t} \qquad \frac{\Delta;\Gamma \vdash e : \text{bag int}}{\Delta;\Gamma \vdash \text{sum } e : \text{int}}$$

$$\frac{\Delta;\Gamma \vdash e : \text{set int}}{\Delta;\Gamma \vdash \text{min } e : \text{option int}} \qquad \frac{\Delta;\Gamma \vdash e : \text{list } t \qquad \Delta;\Gamma, x : t \vdash p : \text{bool}}{\text{filter } e\ (x \to p) : \text{list } t}$$

$$\frac{\begin{array}{c}\Delta;\Gamma \vdash e_1 : \text{list } t_1 \\ \Delta;\Gamma \vdash e_2 : \text{list } t_2 \qquad \Delta;\Gamma, x_1 : t_1, x_2 : t_2 \vdash p : \text{bool} \qquad \Delta;\Gamma, x_1 : t_1, x_2 : t_2 \vdash e_3 : t_3\end{array}}{\text{join } e_1\ e_2\ (x_1, x_2 \to e_3 \text{ if } p) : \text{list } t_3}$$

$$\frac{\Delta;\Gamma \vdash e_1 : \text{list } t_1 \qquad \Delta;\Gamma, x : t_1 \vdash e_2 : t_2}{\text{proj } e_1\ (x \to e_2) : \text{list } t_2}$$

Fig. 4. Typing judgment on Conquord expressions

$$\frac{}{\Delta;\Gamma \vdash \text{skip}} \qquad \frac{\Delta;\Gamma \vdash c_1 \qquad \Delta;\Gamma \vdash c_2}{\Delta;\Gamma \vdash c_1; c_2} \qquad \frac{\Delta;\Gamma \vdash e : t \qquad \Delta, \mathbf{x} : t;\Gamma \vdash c}{\Delta;\Gamma \vdash \text{letmut } \mathbf{x} \coloneqq e \text{ in } c}$$

$$\frac{\Delta[\mathbf{x}] = t \qquad \Delta;\Gamma \vdash e : t}{\Delta;\Gamma \vdash \mathbf{x} \coloneqq e} \qquad \frac{\Delta;\Gamma \vdash e : \text{bool} \qquad \Delta;\Gamma \vdash c_1 \qquad \Delta;\Gamma \vdash c_2}{\Delta;\Gamma \vdash \text{if } e \text{ then } c_1 \text{ else } c_2}$$

$$\frac{\Delta;\Gamma \vdash e : \text{list } t \qquad \Delta;\Gamma, x : t \vdash c}{\Delta;\Gamma \vdash \text{foreach } x \leftarrow e \text{ do } c \text{ end}}$$

Fig. 5. Typing judgment on Conquord commands

The semantics of Conquord expressions are defined by an interpretation function that maps expressions to values. Let $[\![e]\!]_{\phi,\psi}$ denote the interpretation of the expression e in the value environment ϕ for mutable variables and ψ for immutable variables. We formalized and proved the type-soundness theorem for Conquord as stated in Theorem 3.1 in the Rocq theorem prover.

List	L	::=	$[] \mid v :: L$
Bag	B	::=	empty-bag $\mid$ bag-insert $B\ v$
Set	S	::=	empty-set $\mid$ set-insert $S\ v$
Dictionary	D	::=	empty-dict $\mid D[v \mapsto v]$
Value	v	::=	$n \mid b \mid s \mid$ none $\mid$ some $v \mid \{\text{tag} = v; \ldots\} \mid () \mid D \mid L \mid B \mid S$

Fig. 6. Values for defining the semantics of Conquord

$$\frac{}{\vdash n : \text{int}} \qquad \frac{}{\vdash b : \text{bool}} \qquad \frac{}{\vdash s : \text{string}} \qquad \frac{\vdash v : t}{\vdash \text{some } v : \text{option } t} \qquad \frac{}{\vdash \text{none} : \text{option } t}$$

$$\frac{\forall k\ v.\ D[k] = v \implies \vdash k : kt \wedge \vdash v : vt}{\vdash D : \text{dict } kt\ vt}$$

Fig. 7. Typing judgment on Conquord values

Theorem 3.1 (Type soundness for expressions). *For any typing environments* Δ *and* Γ*, Conquord expression* e*, and Conquord type* t*,* $\Delta; \Gamma \vdash e : t \implies \forall \phi\ \psi.\ \Delta \vdash \phi \wedge \Gamma \vdash \psi \implies \vdash [\![e]\!]_{\phi,\psi} : t$.

Similarly, $[\![c]\!]_{\phi,\psi}$ denotes the interpretation of the command c in the value environments, and it evaluates to a value environment for mutable variables after executing c from a state with ϕ and ψ. Theorem 3.2 is also formalized and proven in Rocq.

Theorem 3.2 (Type soundness for commands). *For any typing environments* Δ *and* Γ *and Conquord command* c*,* $\Delta; \Gamma \vdash c \implies \forall \phi\ \psi.\ \Delta \vdash \phi \wedge \Gamma \vdash \psi \implies \Delta \vdash [\![c]\!]_{\phi,\psi}$.

4 REWRITES FOR IDENTIFYING RELATIONAL-ALGEBRA OPERATIONS

A Conquord program expresses a relational database table as a list of records. Relational operations on a table are written as query comprehensions. Query comprehensions are notation for the list monad, with flatmap as the bind operator, so are readily converted to flatmap operations:

$$x \leftarrow e_1; e_2 \Rightarrow \text{flatmap } e_1\ (x \rightarrow e_2)$$
$$\text{check } e_1; e_2 \Rightarrow \text{if } e_1 \text{ then } e_2 \text{ else } []$$
$$\text{ret } e \Rightarrow [e]$$

Applying the rules to the following expression,

$$x \leftarrow \textbf{tbl};$$
$$\text{check } (cond);$$
$$\text{ret } e$$

we obtain

$$\text{flatmap } \textbf{tbl}\ (x \rightarrow \text{if } cond \text{ then } [e] \text{ else } [])$$

Our compiler needs to apply database-style optimizations so it first rewrites expressions into their relational-algebra equivalents. This rewriting is incomplete—not all expressions equivalent to relational-algebra operations will be recognized as such. After this transformation, typical rewrite rules in relational algebra such as filter pushdown can be applied, and the reasoning about the correctness of index-related transformations becomes more straightforward. Figure 8 shows some

rewrite rules transforming expressions that may result from parsing query comprehensions into relational-algebra constructs.

$$\frac{\Delta;\Gamma \vdash e : \text{list } t \qquad \Delta;\Gamma, x : t \vdash p : \text{bool}}{\text{flatmap}_{\text{list}}\ e\ (x \to \text{if } p \text{ then } [x] \text{ else } []) = \text{filter}_{\text{list}}\ e\ (x \to p)}$$

$$\frac{\Delta;\Gamma \vdash e_1 : \text{list } t_1 \qquad \Delta;\Gamma, x : t_1 \vdash e_2 : t_2}{\text{flatmap}_{\text{list}}\ e_1\ (x \to [e_2]) = \text{proj}_{\text{list}}\ e_1\ (x \to e_2)}$$

$$\frac{\begin{array}{c}\Delta;\Gamma \vdash e_1 : \text{list } t_1 \qquad \Delta;\Gamma \vdash e_2 : \text{list } t_2 \\ \Delta;\Gamma, x_1 : t_1, x_2 : t_2 \vdash p : \text{bool} \qquad \Gamma, x_1 : t_1, x_2 : t_2 \vdash e_3 : t_3 \qquad x_1 \neq x_2 \qquad x_1 \notin \text{free-immuts}(e_2)\end{array}}{\text{flatmap}_{\text{list}}\ e_1\ (x_1 \to \text{flatmap}_{\text{list}}\ e_2\ (x_2 \to \text{if } p \text{ then } [e_3] \text{ else } [])) = \text{join}_{\text{list}}\ e_1\ e_2\ (x_1, x_2 \to e_3 \text{ if } p)}$$

Fig. 8. Rules for identifying relational-algebra constructs in Conquord programs

While the rules are written as equations, the compiler applies them from left to right.
We define the soundness of a transformation by Definitions 4.1 and 4.2.

Definition 4.1 (Soundness of expression transformations). A transformation f : expr $\to$ expr is sound if and only if $\forall \Delta\ \Gamma\ e\ t.\ \Delta;\Gamma \vdash e : t \implies \Delta;\Gamma \vdash fe : t \wedge \forall \phi\ \psi.\ \Delta \vdash \phi \implies \Gamma \vdash \psi \implies [\![fe]\!]_{\phi,\psi} = [\![e]\!]_{\phi,\psi}$.

Definition 4.2 (Soundness of command transformations). A transformation f : command $\to$ command is sound if and only if $\forall \Delta\ \Gamma\ c.\ \Delta;\Gamma \vdash c \implies \Delta;\Gamma \vdash fc \wedge \forall \phi\ \psi.\ \Delta \vdash \phi \implies \Gamma \vdash \psi \implies [\![fc]\!]_{\phi,\psi} = [\![c]\!]_{\phi,\psi}$.

All the transformations presented as equations in this paper have soundness proofs checked in Rocq.

Theorem 4.3. *All transformations in Figure 8 are sound.*

5 REWRITES FOR RELAXING COLLECTION CONSTRAINTS

Conquord supports three collection types: lists, bags, and sets. These collections form a hierarchy: lists preserve order and duplicates, bags preserve duplicates but not order, and sets preserve neither. Some Conquord operations are invariant to order, duplicates, or both, which gives us the flexibility to change collection types without changing program semantics. We capture the requirements on collection variables by eagerly introducing collection conversions and pushing these conversion operations into expressions. Index-introduction rules use these conversion operators to recognize when an index representation of a table can be consumed safely.

5.1 Collection Annotation

After performing the relational-algebra rewrites, the compiler introduces conversions to collections that are less restricted than lists when conversion would not change the semantics of the consuming operation. The rewrite rules are shown in Figure 9, and the compiler performs the rewrites from left to right.

5.2 Collection Pushdown

The annotations of bags and sets are pushed down into the subexpressions through the rewrite rules in Figure 10.

$$\frac{\Delta;\Gamma \vdash e : \text{list } t}{\text{sort}_{\text{list}}\ e = \text{sort}_{\text{bag}}\ (\text{bag-of}\ e)} \qquad \frac{\Delta;\Gamma \vdash e : \text{list int}}{\text{fold}\ e\ 0\ (v, a \to (v + a)) = \text{sum}\ (\text{bag-of}\ e)}$$

$$\frac{\Delta;\Gamma \vdash e : \text{list } t}{\text{length}\ e = \text{count}\ (\text{bag-of}\ e)}$$

$$\frac{\Delta;\Gamma \vdash e : \text{list int}}{\begin{array}{c}\text{fold}\ e\ \text{none}\ (v, a \to \text{match}\ a\ \text{with none} \to \text{some}\ v \mid \text{some}\ v' \to \text{if}\ v < v'\ \text{then some}\ v\ \text{else}\ a\ \text{end}) \\ = \text{min}\ (\text{set-of}\ e)\end{array}}$$

Fig. 9. Rules for identifying less-restricted interpretations of collections

$$\frac{\Delta;\Gamma \vdash e_1 : \text{list } t_1 \qquad \Delta;\Gamma, x : t_1 \vdash e_2 : \text{list } t_2}{\text{bag-of}\ (\text{flatmap}\ e_1\ (x \to e_2)) = \text{flatmap}_{\text{bag}}\ (\text{bag-of}\ e_1)\ (x \to (\text{bag-of}\ e_2))}$$

$$\frac{\begin{array}{c}\Delta;\Gamma \vdash e_1 : \text{list } t_1 \\ \Delta;\Gamma \vdash e_2 : \text{list } t_2 \qquad \Delta;\Gamma, x_1 : t_1, x_2 : t_2 \vdash p : \text{bool} \qquad \Delta;\Gamma, x_1 : t_1, x_2 : t_2 \vdash e_3 : t_3\end{array}}{\text{bag-of}\ (\text{join}\ e_1\ e_2\ (x_1, x_2 \to e_3\ \text{if}\ p)) = \text{join}_{\text{bag}}\ (\text{bag-of}\ e_1)\ (\text{bag-of}\ e_2)\ (x_1, x_2 \to e_3\ \text{if}\ p)}$$

$$\frac{\Delta;\Gamma \vdash e_1 : \text{list } t_1 \qquad \Delta;\Gamma, x : t_1 \vdash e_2 : t_2}{\text{bag-of}\ (\text{proj}\ e_1\ (x \to e_2)) = \text{proj}_{\text{bag}}\ (\text{bag-of}\ e_1)\ (x \to e_2)}$$

$$\frac{\Delta;\Gamma \vdash e_1 : t \qquad \Delta;\Gamma \vdash e_2 : \text{list } t}{\text{bag-of}\ (e_1 :: e_2) = \text{insert}_{\text{bag}}\ (\text{bag-of}\ e_2)\ e_1}$$

$$\frac{\Delta;\Gamma \vdash e_1 : \text{list } t_1 \qquad \Delta;\Gamma, x : t_1 \vdash e_2 : \text{list } t_2}{\text{set-of}\ (\text{flatmap}\ e_1\ (x \to e_2)) = \text{flatmap}_{\text{set}}\ (\text{set-of}\ e_1)\ (x \to (\text{set-of}\ e_2))}$$

$$\frac{\begin{array}{c}\Delta;\Gamma \vdash e_1 : \text{list } t_1 \\ \Delta;\Gamma \vdash e_2 : \text{list } t_2 \qquad \Delta;\Gamma, x_1 : t_1, x_2 : t_2 \vdash p : \text{bool} \qquad \Delta;\Gamma, x_1 : t_1, x_2 : t_2 \vdash e_3 : t_3\end{array}}{\text{set-of}\ (\text{join}\ e_1\ e_2\ (x_1, x_2 \to e_3\ \text{if}\ p)) = \text{join}_{\text{set}}\ (\text{set-of}\ e_1)\ (\text{set-of}\ e_2)\ (x_1, x_2 \to e_3\ \text{if}\ p)}$$

$$\frac{\Delta;\Gamma \vdash e_1 : \text{list } t_1 \qquad \Delta;\Gamma, x : t_1 \vdash e_2 : t_2}{\text{set-of}\ (\text{proj}\ e_1\ (x \to e_2)) = \text{proj}_{\text{set}}\ (\text{set-of}\ e_1)\ (x \to e_2)}$$

$$\frac{\Delta;\Gamma \vdash e_1 : t \qquad \Delta;\Gamma \vdash e_2 : \text{list } t}{\text{set-of}\ (e_1 :: e_2) = \text{insert}_{\text{set}}\ (\text{set-of}\ e_2)\ e_1}$$

Fig. 10. Rules for pushdown of collection casts

Theorem 5.1. *All transformations in Figure 9 and Figure 10 are sound.*

6 REWRITES FOR INDEX CREATION AND USE

This section uses a mix of Gallina (Rocq's dependently typed functional language) and Conquord syntax as Gallina tokens are used to abstract over Conquord expressions. Gallina expressions are colored and underlined to avoid confusion.

The indexes implemented in the Conquord compiler implement a common index interface defined as a Gallina type class in Figure 11. The to-idx[.] member of the type class is a context that takes a Conquord expression e and returns a Conquord expression that computes the index from e. is-tbl-ty takes a type t and returns a Boolean value. If it returns true for a type t, then for any Conquord expression e with $\Delta; \Gamma \vdash e : t$, to-idx$[e]$ correctly computes the index from the table that e evaluates to, and $\Delta; \Gamma \vdash$ to-idx$[e]$: idx-ty t.

$$
\begin{array}{lcl}
\text{to-idx}[.] & : & \text{expr} \to \text{expr} \\
\text{is-tbl-ty} & : & \text{type} \to \text{bool} \\
\text{idx-ty} & : & \text{type} \to \text{type} \\
\text{idx-wf} & : & \text{value} \to \text{value} \to \text{Prop} \\
\text{idx-ty-wf} & : & \forall t.\ \vdash t \implies \text{is-tbl-ty}\ t = \text{true} \implies \vdash \text{idx-ty}\ t \\
\text{to-idx-ty} & : & \forall t.\ \vdash t \implies \text{is-tbl-ty}\ t = \text{true} \implies \epsilon; x : t \vdash \text{to-idx}[x] : \text{idx-ty}\ t \\
\text{to-idx-wf} & : & \forall t.\ \vdash t \implies \text{is-tbl-ty}\ t = \text{true} \implies \vdash v : t \implies \text{idx-wf}\ v\ [\![\text{to-idx}[x]_{\epsilon;x:v}]\!]
\end{array}
$$

Fig. 11. Gallina type class defining the index interface

6.1 Index Creation

A whole-program transformation is used to create an index for a table while preserving the semantics of the entire program. For index creation, we have the local rewrites that transform the expressions whose values are written into the table. The transformations rely on an index interface to abstract away the implementation of the index.

$$\text{letmut } \mathbf{tbl} := e \text{ in } c \Rightarrow \text{letmut } \mathbf{tbl} := \{\ \text{id} = e;\ \text{aux} = \text{to-idx}[e]\ \} \text{ in } c \tag{1}$$

$$\mathbf{tbl} := e \Rightarrow \mathbf{tbl} := \{\ \text{id} = e;\ \text{aux} = \text{to-idx}[e]\ \} \tag{2}$$

To obtain semantic equivalence, reads from the mutable variable **tbl** are transformed in tandem.

$$\mathbf{tbl} \Rightarrow \mathbf{tbl}.\text{id} \tag{3}$$

Theorem 6.1. *A sound transformation is formed by the following steps: take a command of the form letmut* ***tbl*** $:= e$ *in c where e has the type t satisfying* is-idx-ty t = *true in the current typing environements; apply Transformation 1 to letmut* ***tbl*** $:= e$ *in c; apply Transformations 2 and 3 in c wherever* ***tbl*** *is not shadowed.*

To use the created index, the compiler rewrites expressions that contain **tbl**.id to read from **tbl**.aux instead. What expressions may be rewritten depends on the index instance.

6.2 Index-Specific Transformations

The equation for the most basic transformation common to all indexes has the following form.

$$\text{to-idx}[\mathbf{tbl}.\text{id}] = \mathbf{tbl}.\text{aux}$$

Rewriting from the left to the right of the equation, the transformation identifies an expression equivalent to the creation of the index and replaces it by an expression that retrieves the index, which has been created and stored in a mutable variable.

Another common category of transformations identify expressions that are equivalent to updates to existing indices. The expression for index update, however, will depend on the definition of to-idx. The rest of this section presents a few instances of the index interface and the transformations using the indexes.

6.2.1 A Dictionary-Based Index. Suppose attr is an attribute of a table. We have implemented a dictionary-based index that maps each value of attr to the bag of tuples in the table with that attribute value. Using bags instead of lists as index lookup results will give more flexibility in the compilation of the index to a lower-level language.

$$\text{dict-update}[d, k, v] \triangleq$$
$$\text{dict-insert } d\ (k \mapsto \text{insert}_{\text{bag}}\ (\text{match (dict-lookup } d\ k) \text{ with none} \longrightarrow \text{empty}_{\text{bag}} \mid \text{some } b \longrightarrow b \text{ end})\ v)$$
$$\text{dict-idx.to-idx}[x] \triangleq \text{fold } x \text{ empty-dict } (tup, acc \longrightarrow \text{dict-update}[acc, tup.\text{attr}, tup])$$

Given the definition above, the following rewrite rule can change the computation of dict-idx.to-idx$[e_1 :: e_2]$ to an update to dict-idx.to-idx$[e_2]$ using e_1. In the case where $e_2 \equiv$ **tbl**, the previous rewrite rule can be applied, and the resulting program uses **tbl**.aux instead of computing the dictionary-based index from scratch. For brevity, we omit the typing judgment for this rewrite rule.

$$\text{dict-idx.to-idx}[e_1 :: e_2] = \text{let } y \leftarrow \text{dict-idx.to-idx}[e_2] \text{ in } \text{dict-update}[y, e_1.\text{attr}, e_1]$$

where $y \notin \text{free-immuts}(e_1)$.

Moreover, we also have an ad-hoc rewrite rule unique to the dictionary-based index.

$$\text{filter}_{\text{bag}}\ (\text{bag-of } \textbf{tbl}.\text{id})\ (x \longrightarrow x.\text{attr} == e) =$$
$$\text{match (dict-lookup } \textbf{tbl}.\text{aux } e) \text{ with none} \longrightarrow \text{empty}_{\text{bag}} \mid \text{some } b \longrightarrow b \text{ end}$$

where $x \notin \text{free-immuts}(e)$.

6.2.2 Aggregations. Figure 9 presented some rewrite rules that identify aggregation operations like min and sum. We have implemented indexes to cache the aggregation results and perform incremental updates when a new tuple is inserted into the table. Unlike the dictionary-based index where the transformation that uses the index involves a lookup operation on the index, the use of the two aggregation indexes happens to coincide with the transformations that identify the index creation. In other words, "looking up" a sum aggregation index is the same as retrieving the index itself, which evaluates to the cached aggregation result.

Like before, we suppose attr is an attribute of the table in question and define aggregations over that attribute. For sum aggregation over the attribute attr of the table,

$$\text{sum-agg.to-idx}[e] \triangleq \text{sum } (\text{proj}_{\text{bag}}\ (\text{bag-of } e)\ (x \longrightarrow x.\text{attr}))$$

The following transformation identifies an update to the sum aggregation result

$$\text{sum-agg.to-idx}[e_1 :: e_2] = (\text{let } x \leftarrow e_1 \text{ in } x.\text{attr}) + \text{sum-agg.to-idx}[e_2]$$

The minimum aggregation is slightly more complicated, as it returns an option value to handle the case of aggregating over the empty set.

$$\text{min-agg.to-idx}[e] \triangleq \text{min } (\text{proj}_{\text{set}}\ (\text{bag-of } e)\ (x \longrightarrow x.\text{attr}))$$

$$\begin{aligned}&\underline{\text{min-agg.to-idx}}[e_1 :: e_2] = \\ &\text{let } y \leftarrow (\text{let } x \leftarrow e_1 \text{ in } x.\text{attr}) \text{ in} \\ &\text{match } \underline{\text{min-agg.to-idx}}[e_2] \text{ with} \\ &\mid \text{none} \rightarrow \text{some } y \\ &\mid \text{some } x \rightarrow \text{if } (y < x) \text{ then } (\text{some } y) \text{ else } (\text{some } x) \\ &\text{end}\end{aligned}$$

where $y \neq x$ and $y \notin \text{free-immuts}(e_2)$.

6.2.3 Bitmap Index. We have also implemented a simplified variant of bitmap indexes. Fixing an attribute attr of a table and a constant value c that the attribute may take, the bitmap index consists of two parts: a primary-key index with an automatically generated unique ID for each tuple in the table, and a bitmap where each bit indicates whether *tup*.attr == c for its corresponding tuple *tup*.

The creation of the primary-key index is defined by

$$\underline{\text{pk-idx.to-idx}}[e] \triangleq \text{fold } e \ (0, \text{empty-dict}) \ (tup, (c, d) \rightarrow (c + 1, \text{dict-insert } d \ (c \mapsto tup)))$$

It evaluates to a pair, where the first component is the cardinality of the table, and the second is a dictionary mapping the IDs to the tuples.

The update transformation is

$$\underline{\text{pk-idx.to-idx}}[e_1 :: e_2] = \text{let } (c, d) \leftarrow \underline{\text{pk-idx.to-idx}}[e_2] \text{ in } (c + 1, \text{dict-insert } d \ (c \mapsto e_1))$$

The creation of the bitmap is defined by

$$\underline{\text{bitmap.to-idx}}[e] \triangleq \text{flatmap } e \ (tup \rightarrow [tup.\text{attr} == \text{c}])$$

The update transformation is

$$\underline{\text{bitmap.to-idx}}[e_1 :: e_2] = (e_1.\text{attr} == \text{c}) :: \underline{\text{bitmap.to-idx}}[e_2]$$

The bitmap index can replace filtering operations by the *tup*.attr == c predicate:

$$\begin{aligned}&\underline{\text{use-bitmap}} \ m \ d \triangleq \\ &\text{fold } m \ (0, []) \ (b, (acc \rightarrow \\ &(\text{fst } acc + 1, \\ &\text{if } b \\ &\text{then match (dict-lookup } d \text{ (fst } acc)) \text{ with none} \rightarrow \text{snd } acc \mid \text{some } x \rightarrow x :: \text{snd } acc \text{ end} \\ &\text{else snd } acc))\end{aligned}$$

$$\begin{aligned}&\text{filter } \textbf{tbl}.\text{id } (x \rightarrow (x.\text{attr} == \text{c})) = \\ &\text{snd } (\underline{\text{use-bitmap}}(\textbf{tbl}.\text{aux.bitmap})(\text{snd } (\textbf{tbl}.\text{aux.pk})))\end{aligned}$$

where the primary-key index is stored in **tbl**.aux.pk and the bitmap in **tbl**.aux.bitmap.

While the transformation above does not speed up programs assuming the entire database is available in the memory, it is a step towards the more interesting use of bitmap indexes, where the predicate may be a conjunction or a disjunction of multiple predicates represented by different bitmaps. The filter operation can then be rewritten to bit-vector operations between the bitmaps, and the resulting bitmap can be used to perform lookups in the primary-key index. The bit-vector operations can parallelize much more efficiently than the predicate computation in the original filter operation, thereby speeding up the program execution.

6.2.4 Composite Indexes. In our implementation, the index created in the whole-program transformation is a composite index that consists of an arbitrary number of indexes. In particular, the to-idx expression of the composite index is a record of the to-idx expressions of the constituent indexes, each associated with a unique tag in the record. For example, suppose the composite index $cidx$ contains two indexes idx_1 and idx_2 associated with the tags idx-1 and idx-2 respectively. Then we have

$$cidx.\text{to-idx} \triangleq \{\ \text{idx-1} = idx_1.\text{to-idx};\ \text{idx-2} = idx_2.\text{to-idx}\ \}$$

$$cidx.\text{idx-wf}\ v\ d \triangleq idx_1.\text{idx-wf}\ v\ d.\text{idx-1} \wedge idx_2.\text{idx-wf}\ v\ d.\text{idx-2}$$

and **tbl**.aux.idx-1 can be used in place of **tbl**.aux in the examples above to access the instance of idx_1 stored in **tbl**.

7 EVALUATION

7.1 A Compilation Pipeline

The transformations described in this paper are composed to give a compilation pipeline, which is illustrated in Figure 12. The compiler receives parameters to determine which indexes to create, and it applies transformations related to an index only if that index is created.

The pipeline successfully performed the intended optimizations on a variety of Conquord programs. Using the soundness lemmas of the transformations in the pipeline, we proved the soundness of the compilation pipeline. The proofs that the example programs have the same semantics before and after the compilation follow directly from the soundness of the pipeline.

7.2 Example programs

The first example contains a join operation with an opportunity for the well-known filter-pushdown optimization from relational algebra. The compilation pipeline was able to identify the following subexpression as a join operation and replace the first conjunct of the condition with a filter operation on the first table.

$$\begin{aligned}
&r_1 \leftarrow tbl_1;\\
&r_2 \leftarrow tbl_2;\\
&\text{check } (r_1.\text{A} == 0 \wedge r_1.\text{B} == r_2.\text{B});\\
&\text{ret } \{\ \text{B} = r_1.\text{B}; \text{C} = r_2.\text{C}\ \}
\end{aligned}$$

The following example has a construct similar to a join operation. For each tuple in one table, it iterates through all tuples in the other table and checks the equality on a common attribute. The compilation pipeline successfully identified the filter operation and transformed a program containing this expression to use a dictionary-based index for the filtering.

```
sort_list
(flatmap_list departments (r1 →
  flatmap_list (flatmap_list responses (r2 →
    if (r2.department-id == r1.department-id) then [r2] else [])) (r3 →
      [{ name = r3.name; department = r1.department-name; feedback = r3.feedback }])))
```

The following command is part of a program that contains a foreach-loop to populate a table with information from another table. We passed parameters into the compilation pipeline so that it creates all the different kinds of indexes we described in Section 6 for the table updated by this command. The indexes include a dictionary-based index, a bitmap index, a sum aggregate, and a

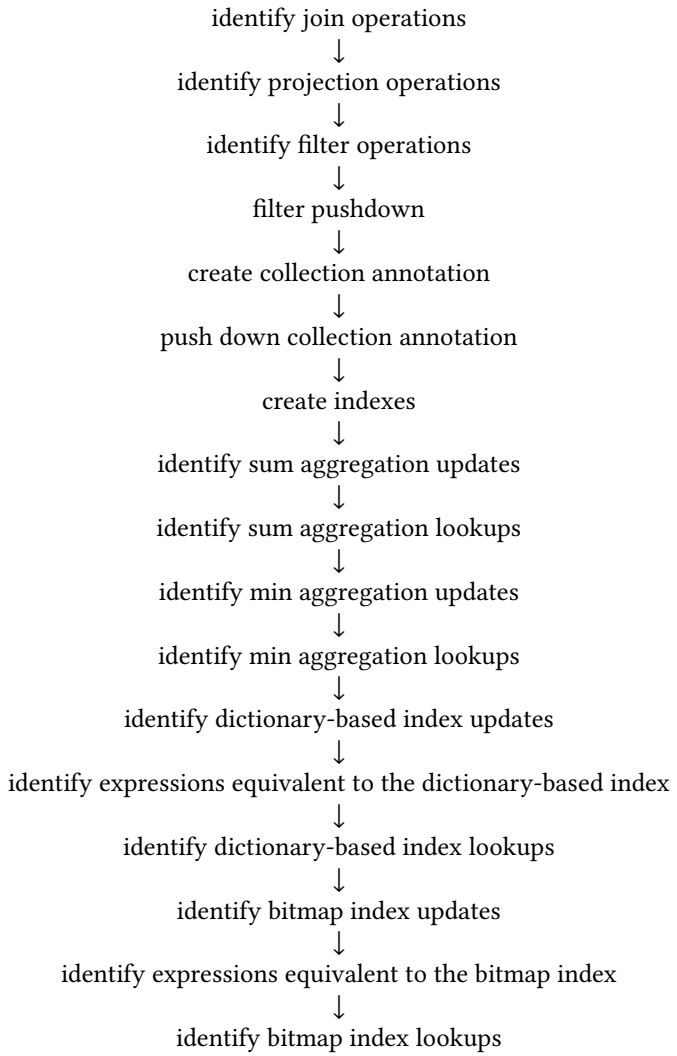


Fig. 12. Example compilation pipeline

minimum aggregate. The resulting program was able to perform incremental updates for all the indexes instead of recomputing them from the updated table.

```
foreach x ← (r ← outfits;
  ret { name = r.shirt-name; color = r.shirt-color; price = r.shirt-price }) do
shirts := x :: shirts end
```

The following command contains an expression equivalent to the minimum aggregate on a table. The compilation pipeline identified the aggregate operation and transformed the expression to the minimum aggregate operation, recognizing that the table can be recognized as a set of tuples under

the aggregate.

```
min-shirt-price :=
fold (r ← shirts; ret r.price) none (v, acc →
match acc with
| none → some v
| some x → if (v < x) then some v else acc
end)
```

Another example contains two subexpressions equivalent to filter operations on different attributes of the same table. The compilation pipeline received parameters to create a dictionary-based index on one attribute and a bitmap index on the other, and both filters were successfully transformed to use the respective indexes.

```
sort_list
(r ← responses;
check (r.department == "CS");
ret r)

sort_list
(r ← responses;
check (r.location == "A");
ret r)
```

Multiple different commands making use of optimizations from the compiler were written together into a single program as shown in Figure 13. The compiler successfully performed the intended optimizations for the respective commands.

The pipeline successfully created the indexes and stored them in the **shirts** variable. In the update-shirts subcommand transformed by the pipeline, all the indexes are incrementally updated, including the minimum aggregation. The white-shirts subexpression is transformed to a lookup into the dictionary-based index, which is instantiated with the shirt colors as the keys. The get-min-price subcommand is also transformed to retrieve the cached minimum aggregate instead of computing it from scratch.

7.3 Lines of Code

Figure 14 shows the number of lines of code in each project file. The statistics are derived using the tool from [12]. A significant amount of effort was devoted to the definitions and the proofs about the language, such as the type system and type soundness. Common utilities in the Utils.v and the TransfUtils.v file also take up a significant proportion of the code size. The number of lines of code in the different index definitions reflect the complexity of the indexes, from 223 lines in SumAgg.v to 1139 lines in BitmapIndex.v.

8 RELATED WORK

Formalized Query Languages. A line of work by Benzaken and Contejean has demonstrated in Coq both a formal semantics of a significant subset of SQL [3] and verified query compilation to imperative code [4]. In this work, we avoid the baggage of literal adherence to SQL standards. Our language is more expressive than traditional query languages in two ways: it includes explicit

```
shirts-row-ty ≜
        { name = string; color = string; price = int }
update-shirts ≜
        foreach x ← (r ← outfits;
          ret { name = r.shirt-name; color = r.shirt-color; price = r.shirt-price }) do
        shirts := x :: shirts end
 white-shirts ≜
        sort_list (r ← shirts; check (r.color == "white"); ret r)
get-shirt-info ≜
        letmut white-shirt-info := "" in
        foreach r ← white-shirts do
        white-shirt-info := r.name ++ ":  " ++
        int-to-string r.price ++ ";  " ++ white-shirt-info end
get-min-price ≜
        min-shirt-price :=
        fold (r ← shirts; ret r.price) none (v, acc →
        match acc with
        | none → some v
        | some x → if (v < x) then some v else acc
        end)
process-shirts ≜
        letmut shirts := []^shirts-row-ty in
        update-shirts; get-shirt-info; get-min-price
```

Fig. 13. Example Conquord program passed into the compilation pipeline

control over data-structure selection, and it includes imperative control flow to allow writing more complex data-processsing applications. This additional flexibility allows us to tackle the orthogonal challenge of integrating SQL-style compilation within a larger compiler for a general-purpose language.

Verified Compilers. Past work on end-to-end-verified computing stacks has often involved high-level languages on top, starting with Lisp in the Computational Logic, Inc. stack [22]. A more recent example is CakeML [19] and its connection to a verified processor [21]. CakeML supports a high-level functional-programming experience, complete with garbage collection. Those functional programs can be written as either normal ML syntax or embedded inside the HOL4 proof assistant, verified in either case, with those results extended into full-stack correctness theorems. We continue this line of work and push the level of abstraction even higher by starting with a language of query comprehensions and making data-structure choices available to the optimizer.

File	blank	comment	code
Utils.v	187	1	1744
TypeSound.v	95	2	1573
TypeSystem.v	47	5	1371
Optimize.v	60	20	1342
BitmapIndex.v	85	0	1139
DictIndexImpl.v	54	2	896
IndexTransf.v	53	0	890
Substitute.v	38	2	733
TransfUtils.v	49	3	693
Value.v	72	2	570
CollectionTransf.v	44	2	555
Interpret.v	28	0	498
ParamPipelineEx.v	60	9	320
TransfSound.v	38	0	298
MinAgg.v	31	0	268
SumAgg.v	31	0	223
RelTransf.v	23	0	219
Notations.v	37	13	205
PipelineEx.v	44	5	193
Language.v	17	11	189
DictIndexImplEx.v	27	5	106
CombIdxEx.v	21	3	83
RelTransfEx.v	13	8	79
SumAggEx.v	19	2	67
IndexInterface.v	3	0	26
SearchCommandDemo.v	17	40	5
SUM:	1193	135	14285

Fig. 14. Lines of code

Relational Query Optimization. The rules we formalize in Section 4 are similar to those used in traditional rule-based SQL optimizers [7, 18]. As in many databases, queries are written in a comprehension syntax similar to the relational calculus [9] and translated to the relational algebra [8] for rewriting. Unlike normal database optimizers, data-structure selection is part of the optimization process, with rules that examine the way that relations are accessed and introduce index structures accordingly.

While data-layout optimization is not usually treated as part of query optimization, the databases community has considered the problem. Work on automatic view and index selection [1, 5, 15, 25, 26] treats index introduction as an optimization problem where extra storage space is traded for increased query performance. This work considers the problem in the context of SQL databases, rather than the more general data-processing-program setting we use, and does not come with guarantees of correctness.

The idea of optimizing a data-processing program by introducing indexes in response to query workloads is not novel, nor are the particular index structures that we introduce. Our contribution lies in the formalization and verification of this process.

Data-Representation Optimization. Prior work in programming languages and compilers has considered the problem of choosing an optimal data representation for a query or collection of queries [14, 17, 20, 30]. This work differs in approach (inductive [17, 20, 30] or deductive [14] program synthesis), expressivity of the query and data-structure language, and the level of automation, but in all cases the result of optimization is either unverified or checked using unsound bounded verification. We show how these sorts of ideas can be fully integrated into end-to-end system verification. By requiring mechanized proofs, we allow for safer component architectures, where tools can be taught new optimizations without endangering soundness.

Fiat [13] considers the problem of data-structure optimization in a verified context, but it implements its compiler transformations as Rocq proof scripts, which makes it particularly difficult to use and reduces performance. We instead deeply embed our query language in Gallina and implement our transformations as Gallina programs with proofs that they are equivalence-preserving. The resulting compiler is efficient and simple to use and extend.

9 CONCLUSION AND FUTURE WORK

The style of transformation we presented here represents the beginning of a potential verified pipeline from very high-level programs to native code with good performance. Most critically, the inference of bag and set semantics and choice of index structures for source-program tables anticipate later compiler phases where we expect our tables to be compiled to corresponding low-level data structures. Our starting point in high-level code with clear semantics of data access may also prove valuable for compiling to concurrent code, ideally implementing classic atomic database transactions. For instance, it may be possible to infer custom locking schemes through static analysis of applications.

So far, we implemented rewriting by directly pattern-matching on ASTs like most compilers, but we plan to investigate the use of e-graphs [23, 28, 31] to do more robust rewriting modulo provable equalities. Since e-graphs remember terms as they are rewritten and select optimal outputs from among them, we hope that they will allow us to use more speculative rewrites for further optimization.

Building verified compilers can be quite labor-intensive, and one established way to divide the work into manageable units is structuring in terms of verified rewrite rules and untrusted heuristics for applying them. We have shown here that such an approach applies more broadly in database-related code than had previously been appreciated.

REFERENCES

[1] Rafi Ahmed, Randall Bello, Andrew Witkowski, and Praveen Kumar. 2020. Automated generation of materialized views in oracle. *Proceedings of the VLDB Endowment* 13, 12 (2020), 3046–3058.

[2] Stéphane Azefack, Kamel Aouiche, and Jérôme Darmont. 2007. Dynamic index selection in data warehouses. In *2007 Innovations in Information Technologies (IIT)*. IEEE, 1–5.

[3] Véronique Benzaken and Évelyne Contejean. 2019. A Coq Mechanised Formal Semantics for Realistic SQL Queries: Formally Reconciling SQL and Bag Relational Algebra. In *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs* (Cascais, Portugal) *(CPP 2019)*. Association for Computing Machinery, New York, NY, USA, 249–261. https://doi.org/10.1145/3293880.3294107

[4] Véronique Benzaken, Évelyne Contejean, Mohammed Houssem Hachmaoui, Chantal Keller, Louis Mandel, Avraham Shinnar, and Jérôme Siméon. 2022. Translating Canonical SQL to Imperative Code in Coq. *Proc. ACM Program. Lang.* 6, OOPSLA1, Article 83 (apr 2022), 27 pages. https://doi.org/10.1145/3527327

[5] Nicolas Bruno and Surajit Chaudhuri. 2005. Automatic Physical Database Tuning: A Relaxation-based Approach. In *Proceedings of the ACM SIGMOD International Conference on Management of Data, Baltimore, Maryland, USA, June 14-16, 2005*, Fatma Özcan (Ed.). ACM, 227–238. https://doi.org/10.1145/1066157.1066184

[6] P. Buneman, L. Libkin, D. Suciu, V. Tannen, and L. Wong. 1994. Comprehension Syntax. *SIGMOD Record* 23, 1 (March 1994), 87–96. https://doi.org/10.1145/181550.181564

[7] Surajit Chaudhuri. 1998. An Overview of Query Optimization in Relational Systems. In *Proceedings of the Seventeenth ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems, June 1-3, 1998, Seattle, Washington, USA*, Alberto O. Mendelzon and Jan Paredaens (Eds.). ACM Press, 34–43. https://doi.org/10.1145/275487.275492

[8] E. F. Codd. 1970. A relational model of data for large shared data banks. *Commun. ACM* 13, 6 (June 1970), 377–387. https://doi.org/10.1145/362384.362685

[9] E. F. Codd. 1971. A Database Sublanguage Founded on the Relational Calculus. In *Proceedings of 1971 ACM-SIGFIDET Workshop on Data Description, Access and Control, San Diego, California, USA, November 11-12, 1971*, E. F. Codd and A. L. Dean (Eds.). ACM, 35–68.

[10] William Cook and Ali Ibrahim. 2005. Integrating Programming Languages & Databases: What's the Problem? (01 2005).

[11] Ezra Cooper, Sam Lindley, Philip Wadler, and Jeremy Yallop. 2006. Links: Web programming without tiers. In *International Symposium on Formal Methods for Components and Objects*. Springer, 266–296.

[12] Albert Danial. 2025. *cloc: v2.06*. https://doi.org/10.5281/zenodo.15734241

[13] Benjamin Delaware, Clément Pit-Claudel, Jason Gross, and Adam Chlipala. 2015. Fiat: Deductive Synthesis of Abstract Data Types in a Proof Assistant. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2015, Mumbai, India, January 15-17, 2015*, Sriram K. Rajamani and David Walker (Eds.). ACM, 689–700. https://doi.org/10.1145/2676726.2677006

[14] John Feser, Sam Madden, Nan Tang, and Armando Solar-Lezama. 2020. Deductive optimization of relational data storage. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (2020), 1–30.

[15] Himanshu Gupta, Venky Harinarayan, Anand Rajaraman, and Jeffrey D. Ullman. 1997. Index Selection for OLAP. In *Proceedings of the Thirteenth International Conference on Data Engineering, April 7-11, 1997, Birmingham, UK*, W. A. Gray and Per-Åke Larson (Eds.). IEEE Computer Society, 208–219. https://doi.org/10.1109/ICDE.1997.581755

[16] Surabhi Gupta and Karthik Ramachandra. 2021. Procedural extensions of SQL: understanding their usage in the wild. *Proc. VLDB Endow.* 14, 8 (April 2021), 1378–1391. https://doi.org/10.14778/3457390.3457402

[17] Peter Hawkins, Alex Aiken, Kathleen Fisher, Martin C. Rinard, and Mooly Sagiv. 2011. Data representation synthesis. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2011, San Jose, CA, USA, June 4-8, 2011*, Mary W. Hall and David A. Padua (Eds.). ACM, 38–49. https://doi.org/10.1145/1993498.1993504

[18] Matthias Jarke and Jürgen Koch. 1984. Query Optimization in Database Systems. *ACM Comput. Surv.* 16, 2 (1984), 111–152. https://doi.org/10.1145/356924.356928

[19] Ramana Kumar, Magnus O. Myreen, Michael Norrish, and Scott Owens. 2014. CakeML: A Verified Implementation of ML. In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Diego, California, USA) *(POPL '14)*. Association for Computing Machinery, New York, NY, USA, 179–191. https://ts.data61.csiro.au/publications/nicta_full_text/7494.pdf 10.1145/2535838.2535841.

[20] Calvin Loncaric, Michael D. Ernst, and Emina Torlak. 2018. Generalized data structure synthesis. In *Proceedings of the 40th International Conference on Software Engineering, ICSE 2018, Gothenburg, Sweden, May 27 - June 03, 2018*, Michel Chaudron, Ivica Crnkovic, Marsha Chechik, and Mark Harman (Eds.). ACM, 958–968. https://doi.org/10.1145/3180155.3180211

[21] Andreas Lööw, Ramana Kumar, Yong Kiam Tan, Magnus O. Myreen, Michael Norrish, Oskar Abrahamsson, and Anthony C. J. Fox. 2019. Verified compilation on a verified processor. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2019, Phoenix, AZ, USA, June 22-26, 2019*, Kathryn S. McKinley and Kathleen Fisher (Eds.). ACM, 1041–1053. https://doi.org/10.1145/3314221.3314622

[22] Strother J. Moore. 2002. A Grand Challenge Proposal for Formal Methods: A Verified Stack. *10th Anniversary Colloquium of UNU/IIST* (2002), 161–172. https://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.7.8607&rep=rep1&type=pdf

[23] Greg Nelson and Derek C. Oppen. 1980. Fast Decision Procedures Based on Congruence Closure. *J. ACM* 27, 2 (April 1980), 356–364. https://doi.org/10.1145/322186.322198

[24] Wilmer Ricciotti and James Cheney. 2022. A Formalization of SQL with Nulls. *Journal of Automated Reasoning* 66, 4 (2022), 989–1030.

[25] Michael Stonebraker. 1974. The choice of partial inversions and combined indices. *International Journal of Parallel Programming* 3, 2 (1974), 167–188. https://doi.org/10.1007/BF00976642

[26] Zohreh Asgharzadeh Talebi, Rada Chirkova, Yahya Fathi, and Matthias F. Stallmann. 2008. Exact and inexact methods for selecting views and indexes for OLAP performance improvement. In *EDBT 2008, 11th International Conference on Extending Database Technology, Nantes, France, March 25-29, 2008, Proceedings (ACM International Conference Proceeding Series, Vol. 261)*, Alfons Kemper, Patrick Valduriez, Noureddine Mouaddib, Jens Teubner, Mokrane Bouzeghoub, Volker Markl, Laurent Amsaleg, and Ioana Manolescu (Eds.). ACM, 311–322. https://doi.org/10.1145/1353343.1353383

[27] P. Trinder and P. Wadler. 1989. Improving list comprehension database queries. In *Fourth IEEE Region 10 International Conference TENCON*. 186–192. https://doi.org/10.1109/TENCON.1989.176921

[28] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. Egg: Fast and Extensible Equality Saturation. *Proc. ACM Program. Lang.* 5, POPL, Article 23 (Jan. 2021), 29 pages. https://doi.org/10.1145/3434304

[29] Sai Wu, Ying Li, Haoqi Zhu, Junbo Zhao, and Gang Chen. 2022. Dynamic index construction with deep reinforcement learning. *Data Science and Engineering* 7, 2 (2022), 87–101.

[30] Cong Yan and Alvin Cheung. 2019. Generating Application-specific Data Layouts for In-memory Databases. *Proc. VLDB Endow.* 12, 11 (2019), 1513–1525. https://doi.org/10.14778/3342263.3342630

[31] Yihong Zhang, Yisu Remy Wang, Oliver Flatt, David Cao, Philip Zucker, Eli Rosenthal, Zachary Tatlock, and Max Willsey. 2023. Better together: Unifying Datalog and equality saturation. *Proceedings of the ACM on Programming Languages* 7, PLDI (2023), 468–492.